

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 12

дисциплина: Архитектура компьютера

<<Программирование в командном процессоре ОС UNIX>>

Студент: **ИБРАХИМ ХИССЕИН ГАНА**

Группа: **НПИбд 01–25**

МОСКВА

2026 г.

Цель работы

Изучить основы программирования в оболочке bash: переменные, циклы, условия, работа с аргументами, тестирование файлов.

Выполнение работы

1. Создание рабочего каталога

```
mkdir -p ~/lab12
```

```
cd ~/lab12
```

```
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~$ mkdir -p ~/lab12
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~$ cd ~/lab12
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab12$
```

2. Скрипт 1 : резервное копирование самого себя

```
nano backup_self.sh
```

Содержимое backup_self.sh:

```
GNU nano 7.2 backup_self.sh *
#!/bin/bash
mkdir -p ~/backup
tar -czf ~/backup/backup_self.tar.gz "$0"
echo "Sauvegarde de $0 effectuée dans ~/backup/backup_self.tar.gz"
```

Запуск и результат:

```
chmod +x backup_self.sh
```

```
./backup_self.sh
```

```
ls -l ~/backup/
```

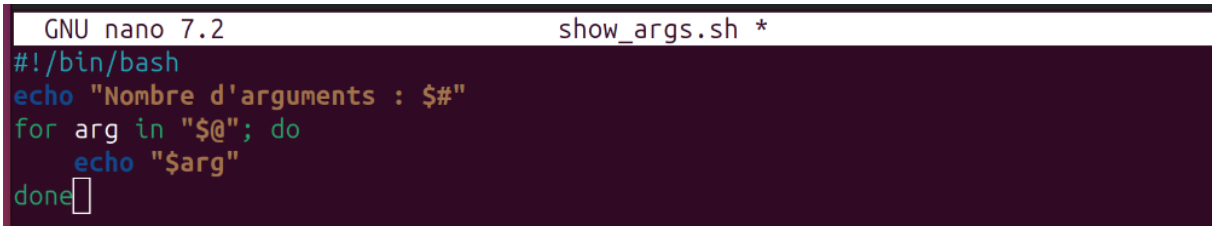
```
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab12$ ./backup_self.sh
Sauvegarde de ./backup_self.sh effectuée dans ~/backup/backup_self.tar.gz
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab12$ ls -l ~/backup/
итого 4
-rw-rw-r-- 1 ibrahim ibrahim 219 mai  2 16:09 backup_self.tar.gz
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab12$
```

3. Скрипт 2 : обработка произвольного числа аргументов

nano show_args.sh

Содержимое show_args.sh:

```
GNU nano 7.2 show_args.sh *
#!/bin/bash
echo "Nombre d'arguments : $#"
```

A screenshot of the nano text editor. The title bar shows 'GNU nano 7.2' and 'show_args.sh *'. The editor content is a bash script: a shebang line '#!/bin/bash', an echo statement 'echo "Nombre d'arguments : \$#"', a for loop 'for arg in "\$@"; do', an indented echo statement 'echo "\$arg"', and a 'done' statement. The cursor is at the end of the 'done' line.

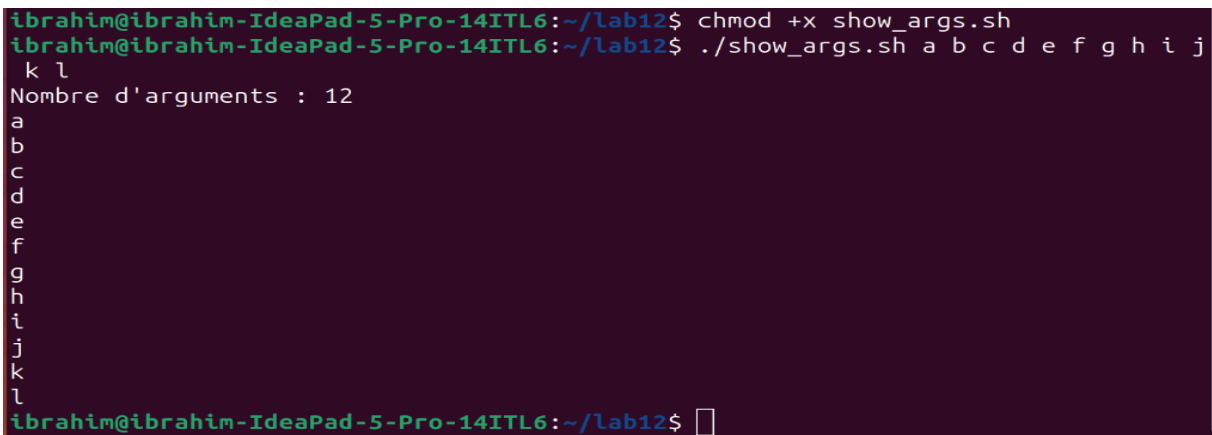
```
    echo "$arg"
done
```

Запуск с 12 аргументами:

chmod +x show_args.sh

./show_args.sh a b c d e f g h i j k l

```
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab12$ chmod +x show_args.sh
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab12$ ./show_args.sh a b c d e f g h i j
k l
Nombre d'arguments : 12
a
b
c
d
e
f
g
h
i
j
k
l
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab12$
```

A screenshot of a terminal window. It shows the execution of the script. The prompt is 'ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab12\$'. The first command is 'chmod +x show_args.sh'. The second command is './show_args.sh a b c d e f g h i j k l'. The output is 'Nombre d'arguments : 12' followed by a vertical list of the arguments 'a' through 'l'. The prompt returns at the end.

```
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab12$
```

4. Скрипт 3 : аналог команды ls (без ls)

nano my_ls.sh

Содержимое my_ls.sh:

```
GNU nano 7.2 my_ls.sh *
#!/bin/bash
dir="${1:-.}"
for file in "$dir"/*; do
    if [ -e "$file" ]; then
        perms=""
        [ -d "$file" ] && perms="${perms}d" || perms="-"
        [ -r "$file" ] && perms="${perms}r" || perms="${perms}-"
        [ -w "$file" ] && perms="${perms}w" || perms="${perms}-"
        [ -x "$file" ] && perms="${perms}x" || perms="${perms}-"
        echo "$perms $(basename "$file")"
    fi
done
```

Запуск для текущего каталога и /etc:

chmod +x my_ls.sh

./my_ls.sh

./my_ls.sh /etc

```
-rwx my_ls.sh
-rwx show_args.sh
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab12$ ./my_ls.sh /etc
-r-- adduser.conf
-r-- aliases
-r-- aliases.db
dr-x alsa
dr-x alternatives
-r-- anacrontab
dr-x apache2
-r-- apg.conf
dr-x apm
dr-x apparmor
dr-x apparmor.d
```

5. Скрипт 4 : подсчёт файлов по расширению

nano count_ext.sh

Содержимое count_ext.sh (до/после исправления):

```
GNU nano 7.2 count_ext.sh *
#!/bin/bash
if [ $# -ne 2 ]; then
    echo "Usage: $0 <extension> <répertoire>"
    exit 1
fi
ext="$1"
dir="$2"
count=$(find "$dir" -type f -name "*$ext") 2>/dev/null | wc -l)
echo "число файл с l'extension $ext dans $dir : $count"
```

Запуск после исправления:

```
chmod +x count_ext.sh
```

```
./count_ext.sh .txt /etc
```

```
./count_ext.sh: строка 8: `count=$(find "$dir" -type f -name "$ext") 2>/dev/nul  
l | wc -l)`  
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab12$ nano count_ext.sh  
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab12$ ./count_ext.sh .txt /etc  
Nombre de fichiers avec l'extension .txt dans /etc : 19  
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab12$
```

Выводы

В ходе работы были написаны и отлажены четыре bash-скрипта:

Резервное копирование самого себя.

Обработка произвольного количества аргументов.

Эмуляция команды ls с отображением прав доступа.

Подсчёт файлов заданного расширения в указанном каталоге.

Получены практические навыки работы с переменными, циклами, условиями, передачей параметров и тестированием файлов в оболочке bash.

Ответы на контрольные вопросы

1. Объясните понятие командной оболочки. Приведите примеры командных оболочек. Чем они отличаются?

Командная оболочка (shell) — программа, обеспечивающая интерфейс между пользователем и ОС. Примеры: sh (Bourne shell), bash (Bourne again shell), csh, zsh. Отличаются синтаксисом, возможностями (история, автодополнение, скрипты).

2. Что такое POSIX?

POSIX — набор стандартов, описывающих интерфейсы между ОС и прикладными программами, обеспечивающий переносимость на уровне исходного кода.

3. Как определяются переменные и массивы в языке программирования bash?

Переменная: `var=value`. Массив: `arr=(val1 val2)`. Доступ: `$var`, `${arr[0]}`.

4. Каково назначение операторов `let` и `read`?

`let` — для арифметических операций. `read` — чтение ввода пользователя в переменные.

5. Какие арифметические операции можно применять в языке программирования bash?

`+`, `-`, `*`, `/`, `%`, `+=`, `-=`, `*=`, `/=`, `++`, `--`.

6. Что означает операция `(( ))`?

`(( ))` — выполнение арифметических операций в стиле C.

7. Какие стандартные имена переменных Вам известны?

`HOME`, `PATH`, `USER`, `SHELL`, `PWD`, `PS1`, `IFS`, `LOGNAME`.

8. Что такое метасимволы?

Символы, имеющие специальное значение для shell: `*`, `?`, `[ ]`, `|`, `&`, `;`, `( )`, `{ }`, `<`, `>`, (пробел), табуляция.

9. Как экранировать метасимволы?

С помощью `\` (обратный слеш) или заключением в одинарные кавычки `' '`.

10. Как создавать и запускать командные файлы?

Создать текстовый файл с `#!/bin/bash`, сделать его исполняемым (`chmod +x`), запускать: `./файл`.

11. Как определяются функции в языке программирования bash?

`function имя { команды; }`

12. Каким образом можно выяснить, является файл каталогом или обычным файлом?

С помощью `test -d` (каталог) или `test -f` (обычный файл).

13. Каково назначение команд `set`, `typeset` и `unset`?

`set` — отображает и устанавливает переменные. `typeset` (или `declare`) — объявляет переменные с атрибутами. `unset` — удаляет переменные.

14. Как передаются параметры в командные файлы?

Параметры передаются через пробел после имени файла. Внутри скрипта доступны как `$1`, `$2`, ..., `$9`, `${10}`, `${11}`...

15. Назовите специальные переменные языка `bash` и их назначение.

`$#` — число аргументов

`$*`, `$@` — все аргументы

`$?` — код возврата последней команды

`$$` — PID оболочки